Heuristic Search

CSI 4106 Introduction to Artificial Intelligence

Marcel Turcotte
Version: Nov 4, 2026 17:10

Preamble

Message of the Day

Learning Objectives

- **Distinguish** the heuristic estimate h, path cost g, and search priority f.
- **Implement** A* graph search using a priority queue and best-known path costs.
- **Explain** cost optimality under admissibility and the stated duplicate-handling policy.
- **Construct and justify** misplaced-tile and Manhattan heuristics for the 8-puzzle.
- **Compare** search effort and memory requirements under controlled conditions.

Python Helpers from Lecture 16

In [1]:
```
import random
from collections import deque

import matplotlib.pyplot as plt
import numpy as np

random.seed(58)

def is_solvable(tiles):
    # Count the inversions in the flattened list of tiles (excluding the bla
    # Assumption: board is square and width is odd.
    inversions = 0
    for i in range(len(tiles)):
        for j in range(i + 1, len(tiles)):
            if tiles[i] != 0 and tiles[j] != 0 and tiles[i] > tiles[j]:
                inversions += 1
    return inversions % 2 == 0

def generate_solvable_board():
    # Generate a random board configuration that is guaranteed to be solvabl
    tiles = list(range(9))
    random.shuffle(tiles)
    while not is_solvable(tiles):
```

```
        random.shuffle(tiles)

    return tuple(tiles)

def plot_board(board, title, num_pos, position):
    ax = plt.subplot(1, num_pos, position)
    ax.set_title(title)
    ax.set_xticks([])
    ax.set_yticks([])

    board = np.array(board).reshape(3, 3).tolist() # Reconfigure the grid to

    # Use a color map to display the numbers
    cmap = plt.cm.plasma
    norm = plt.Normalize(vmin=-1, vmax=8)

    for i in range(3):
        for j in range(3):
            tile_value = board[i][j]
            color = cmap(norm(tile_value))
            ax.add_patch(plt.Rectangle((j, 2 - i), 1, 1, facecolor=color, ec
            if tile_value == 0:
                ax.add_patch(plt.Rectangle((j, 2 - i), 1, 1, facecolor='whit
            else:
                ax.text(j + 0.5, 2 - i + 0.5, str(tile_value),
                        fontsize=16, ha='center', va='center', color='black'

    ax.set_xlim(0, 3)
    ax.set_ylim(0, 3)

def is_empty(frontier):
    """Checks if the frontier is empty."""
    return len(frontier) == 0

def is_goal(state, goal_state):
    """Determines if a given state matches the goal state."""
    return state == goal_state

def expand(state):
    """Generates successor states by moving the blank tile in all possible d
    size = int(len(state) ** 0.5)  # Determine puzzle size (3 for 8-puzzle,
    idx = state.index(0)  # Find the index of the blank tile represented by
    x, y = idx % size, idx // size  # Convert index to (x, y) coordinates
    neighbors = []

    # Define possible moves: Left, Right, Up, Down
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        # Check if the new position is within the puzzle boundaries
        if 0 <= nx < size and 0 <= ny < size:
            n_idx = ny * size + nx
            new_state = list(state)
            # Swap the blank tile with the adjacent tile
            new_state[idx], new_state[n_idx] = new_state[n_idx], new_state[i
            neighbors.append(tuple(new_state))
```

```
    return neighbors

def print_solution(solution):
    """Prints the sequence of steps from the initial to the goal state."""
    size = int(len(solution[0]) ** 0.5)
    for step, state in enumerate(solution):
        print(f"Step {step}:")
        for i in range(size):
            row = state[i*size:(i+1)*size]
            print(' '.join(str(n) if n != 0 else ' ' for n in row))
        print()

def bfs(initial_state, goal_state):

    frontier = deque()  # Initialize the queue for BFS
    frontier.append((initial_state, []))  # A node: (state, preceding states

    reached = {initial_state}

    expanded = 0  # Count non-goal nodes whose successors are generated

    while not is_empty(frontier):
        current_state, path = frontier.popleft()

        if is_goal(current_state, goal_state):
            print(f"Nodes expanded: {expanded}")
            return path + [current_state]  # Return the successful path

        expanded += 1

        for neighbor in expand(current_state):
            if neighbor not in reached:
                reached.add(neighbor)
                frontier.append((neighbor, path + [current_state]))

    return None

def dfs(initial_state, goal_state):

    frontier = [(initial_state, [])]  # A node: (state, preceding states)

    reached = {initial_state}

    expanded = 0

    while not is_empty(frontier):
        current_state, path = frontier.pop()

        if is_goal(current_state, goal_state):
            print(f"Nodes expanded: {expanded}")
            return path + [current_state]  # Return the successful path

        expanded += 1

        for neighbor in expand(current_state):
            if neighbor not in reached:
```

```
                reached.add(neighbor)
                frontier.append((neighbor, path + [current_state]))

    return None
```

These functions were introduced in Lecture 16 and are required to run the examples in Lecture 17. Expand the code to revisit state representation, successor generation, goal tests, and the BFS/DFS baselines.

The problem formulation, search-tree vocabulary, frontier, and BFS/DFS baselines are developed in Lecture 16. The helper code on this slide keeps the notebook independently executable and provides a reference for revisiting those concepts.

A search state is a flat tuple of nine integers, with `0` denoting the blank. Tuples of integers are immutable and hashable, so states can be used directly as set elements or dictionary keys. Only board generation and successor construction use temporary mutable lists; each completed state is returned as a tuple. Frontier containers and paths remain separate from the state representation. A reached-state set records discovered states, including those still on the frontier.

Remarks

- **Breadth-first search (BFS)** identifies the optimal solution, 25 moves, after 145,605 non-goal node expansions.
- **Depth-first search (DFS)** discovers a solution involving 1,157 moves after 1,187 non-goal node expansions.

How can solutions be discovered **more efficiently**?

Informed Search

Heuristic Search

An **informed search** uses an estimate of the **remaining cost to a goal** to guide exploration.

. . .

How could such an estimate help **choose the next frontier node**?

The estimate uses knowledge beyond the problem's actions, transition model, goal test, and action costs. A goal may be a city, a solved puzzle, or a condition on an abstract state; it need not have a physical location.

The estimate guides which possibilities to examine. It does not replace the transition model or verify that a complete solution exists.

Heuristics and Search Priorities

For a search node n representing state s:

- $g(n)$: **known cost of this path** from the initial state to s.
- $h(s)$: **estimated least remaining cost** from s to a goal.
- $f(n)$: **priority** used to select a frontier entry.

Best-first strategy	Priority $f(n)$
Greedy best-first search	$h(s)$
Uniform-cost search	$g(n)$
A* search	$g(n) + h(s)$

Best-first search selects a frontier entry with minimum f. The heuristic h and path cost g must use the same cost units.

A state describes a configuration, while a search node also records a path to it. Two nodes can therefore have the same state and heuristic estimate but different path costs. The notation $h(n)$ is often used as shorthand for $h(s)$, where s is the state stored in n.

Greedy search uses the estimated remaining cost alone and has no general least-cost guarantee. A* also considers the cost already incurred. Uniform-cost search is the special case of A* with $h = 0$. See Berkeley CS 188, Informed Search.

A Route-Finding Heuristic

In a road-distance problem, **straight-line distance** to the destination estimates the remaining distance.

. . .

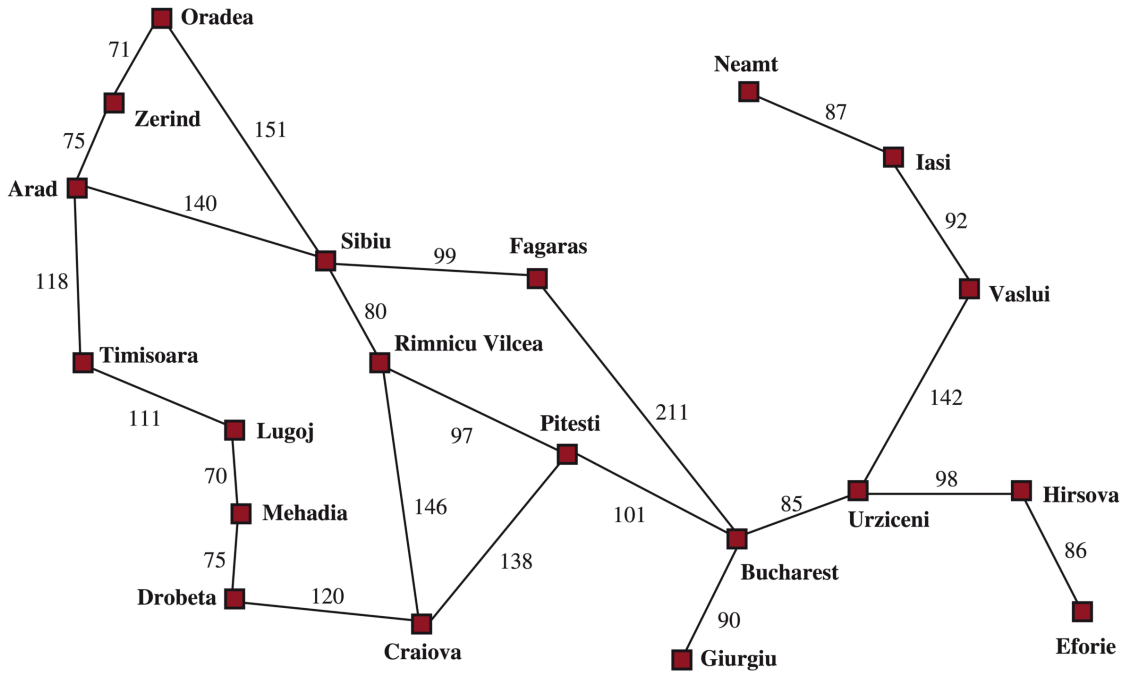
The straight line need **not** be a usable road. A* adds this estimate to the distance already travelled:

$$f(n) = g(n) + h(s).$$

Both terms are measured in kilometres in the following example. Straight-line distance is a lower bound on road distance. It would not directly estimate travel time without a suitable conversion to time units.

The Romania example uses A*: the displayed priorities include both travelled distance and straight-line distance. Selecting the city with the smallest straight-line distance alone would instead be greedy best-first search.

Romania: Route-Finding Problem



Source: (Russell and Norvig 2020, fig. 3.1)

Problem: Find a least-road-distance route from Arad to Bucharest. States are cities, actions follow roads, and action costs are road distances in kilometres.

Romania: Straight-Line Heuristic

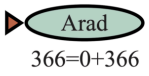
Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Source: (Russell and Norvig 2020, fig. 3.16)

The table supplies straight-line distances to Bucharest. These are heuristic values $h(s)$, not total route costs.

Romania: Initial Frontier

(a) The initial state

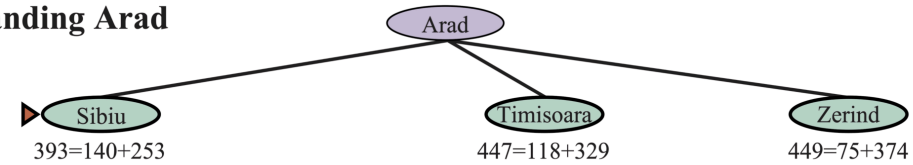


Source: (Russell and Norvig 2020, fig. 3.18)

Arad enters the frontier with $g(n) = 0$ and $h(s) = 366$, so its priority is $f(n) = 366$. Every displayed total in the following figures is a value of $f = g + h$.

Romania: Expand Arad

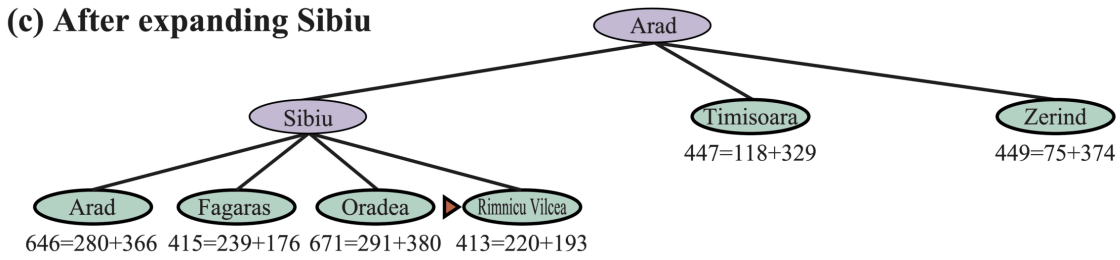
(b) After expanding Arad



Source: (Russell and Norvig 2020, fig. 3.18)

From Arad, the new entries are Sibiu ($140 + 253 = 393$), Timisoara ($118 + 329 = 447$), and Zerind ($75 + 374 = 449$). Sibiu has the smallest f and is selected next.

Romania: Expand Sibiu

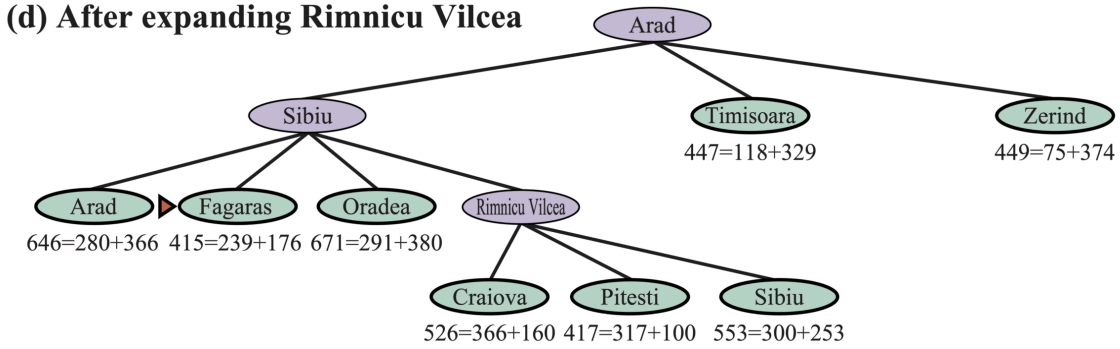


Source: (Russell and Norvig 2020, fig. 3.18)

Expanding Sibiu generates paths to Arad, Fagaras, Oradea, and Rimnicu Vilcea. Rimnicu Vilcea has $g = 220$, $h = 193$, and $f = 413$, so it is selected before Fagaras, whose priority is 415.

These figures retain search-tree occurrences, including repeated cities. The graph-search implementation developed below instead discards dominated paths and keeps a new entry when a city's recorded cost improves.

Romania: Expand Rimnicu Vilcea

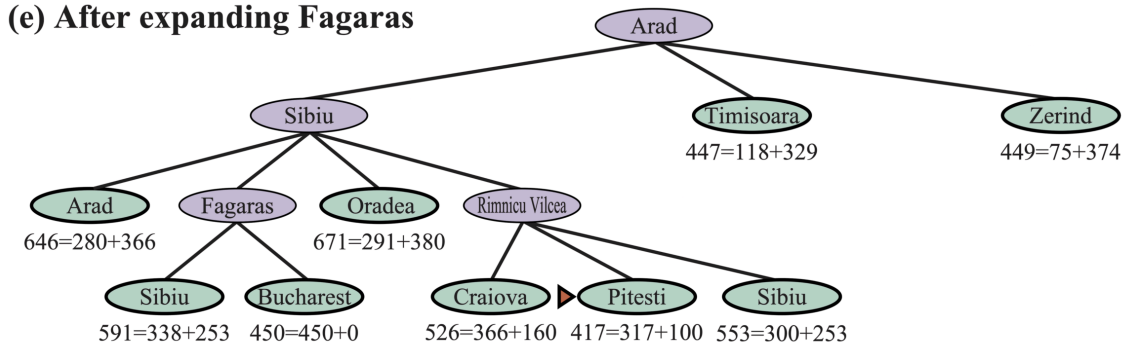


Source: (Russell and Norvig 2020, fig. 3.18)

From Rimnicu Vilcea, the candidates are Craiova ($366 + 160 = 526$), Pitesti ($317 + 100 = 417$), and Sibiu ($300 + 253 = 553$). The second occurrence of Sibiu represents a more expensive path to the same city; its first path cost was 140. A graph-search implementation would discard the cost-300 route to Sibiu.

Which frontier entry is selected next? Fagaras still has the smallest priority, $f = 415$, followed by Pitesti at 417.

Romania: Expand Fagaras



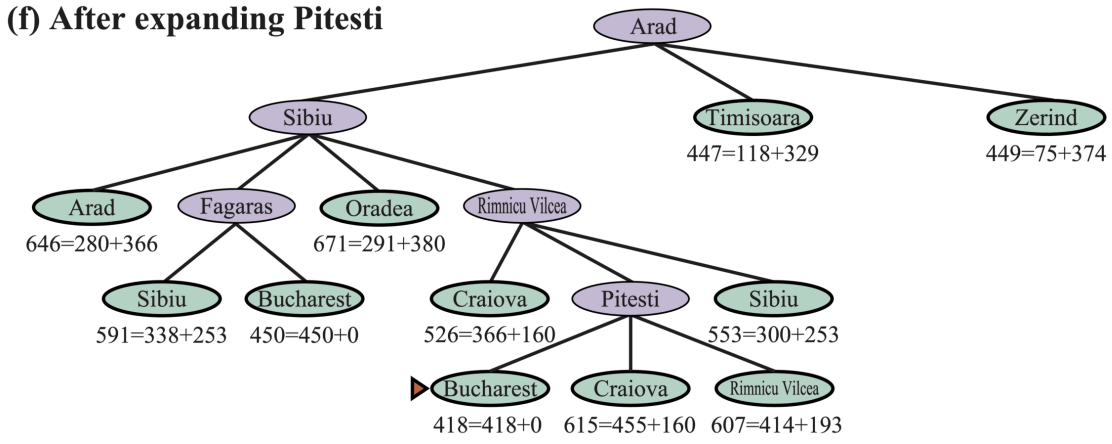
Source: (Russell and Norvig 2020, fig. 3.18)

Expanding Fagaras generates a Bucharest entry with $g = 450$ and $h = 0$, hence $f = 450$. This is a complete route, but generating a goal entry does not end A*.

Pitesti ($f = 417$), Timisoara (447), and Zerind (449) all have lower priority values than this goal entry. Pitesti is selected next.

The goal test is applied when a current minimum-f entry is removed from the frontier. At a goal, $h = 0$ and $f = g$; search does not inherently make the heuristic more accurate as execution progresses.

Romania: Select Bucharest



Source: (Russell and Norvig 2020, fig. 3.18)

Expanding Pitesti generates a cheaper Bucharest entry with cost 418. Since $h(\text{Bucharest}) = 0$, its priority is also 418, lower than those of the other frontier entries.

The returned route is Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest, with total road distance **418 km**. The previous cost-450 Bucharest entry illustrates why an already reached state may need a cheaper path.

Why is selecting this goal sufficient to establish cost optimality? The following implementation and proof state the required conditions.

A Priority Queue

How should the BFS/DFS frontier change when nodes have different priorities?

. . .

Use a **priority queue** ordered by the evaluation function f.

In [2]:
```
import heapq
```

Python's `heapq` removes the smallest entry. For A*, the first component is $g + h$.

BFS as Best-First Search

BFS selects nodes in nondecreasing **depth**. It can be described using the evaluation function

$$f(n) = \text{depth}(n).$$

Depth is known from the path; it is not a heuristic estimate of remaining goal cost. With equal action costs, depth and path cost induce the same ordering between levels. A FIFO queue is a simpler implementation of BFS than a heap and preserves its usual within-level order.

DFS can similarly be described by prioritizing deeper nodes, using negative depth and a suitable last-in, first-out tie rule. The choice of tie rule matters when several entries have the same priority.

A* Search

$$f(n) = g(n) + h(s).$$

- $g(n)$ is the cost of the **particular path** stored in entry n.
- $h(s)$ estimates the **least remaining cost** from its state s.
- A* selects a frontier entry with **minimum** f.

Hart et al. (1968)

Knowing $g(n)$ exactly for a recorded path does not mean that this is the cheapest path to the state. Another route can improve it. The dictionary in the next implementation records the cheapest cost discovered so far.

The 8-puzzle implementation uses unit action costs, so extending a path changes g to $g + 1$. The same idea applies to other cost models by adding the actual action cost.

A* Implementation

In [3]:
```python
def astar(initial_state, goal_state, heuristic):
    # Entries contain (f, g, state, preceding states).
    frontier = [(heuristic(initial_state, goal_state), 0, initial_state, [])
    best_g = {initial_state: 0}
    expanded = 0

    while frontier:
        f, g, current_state, path = heapq.heappop(frontier)
        # A cheaper path may have replaced this queue entry.
        if g > best_g[current_state]:
            continue

        if is_goal(current_state, goal_state):
            return path + [current_state], expanded

        expanded += 1
        for neighbor in expand(current_state):
            new_g = g + 1  # Each puzzle move costs 1.
            if neighbor not in best_g or new_g < best_g[neighbor]:
                best_g[neighbor] = new_g
                new_f = new_g + heuristic(neighbor, goal_state)
                heapq.heappush(frontier, (new_f, new_g, neighbor, path + [cu

    return None, expanded
```

The function returns `(solution_states, expanded)`. A solution includes the initial and goal states, so its number of moves is `len(solution_states) - 1`.

Keeping Better Paths

- `best_g[state]` records the **cheapest path discovered so far**, not a final cost.
- Enqueue a successor whenever its new path cost is **strictly smaller**.
- Skip a popped entry if its stored `g` is larger than `best_g[state]`.
- A cheaper path can **reopen** an already expanded state.
- Test for a goal only after popping a **current minimum-f entry**.

A frontier entry represents a particular path to a state. Several entries may refer to the same state, with different path costs. When a better path is found, the new entry is added; an older, more expensive entry may remain in the heap. The stale-entry check discards it before the goal test or expansion. For example, if a state is queued at cost 7 and later reached at cost 5, `best_g` becomes 5; the old cost-7 entry is skipped when popped.

An expanded state is not permanently closed in this implementation. If its recorded cost improves, its successors must be considered again using that cheaper path. The `expanded` result counts actual non-goal expansions, including any repeated expansions; it does not count stale entries or the selected goal.

For the 8-puzzle, each action costs one, so the new path cost is `g + 1`. With unequal action costs, the same policy uses `g + action_cost` instead.

The best-cost update follows the policy in AIMA, Figure 3.7.

Admissibility

A heuristic is **admissible** if it never overestimates the least remaining solution cost:

$$0 \leq h(s) \leq h^\star(s).$$

Here $h^\star(s)$ is the true least cost from s to a goal.

What could go wrong if the estimate overstates the remaining cost?

An overestimate can make a genuinely cheap solution look unattractive, so A* may return a more expensive solution. Overestimating does not force a wrong answer on every instance; it removes the general guarantee.

At every goal G, admissibility gives $h(G) = 0$, because the remaining cost is zero. This lecture uses nonnegative heuristic values. If no goal is reachable from a state, its true remaining cost is infinite.

The heuristic condition alone is not the whole algorithm: the implementation must also retain improved paths and test a goal at the appropriate point.

Cost Optimality

An algorithm is **cost optimal** when it returns a solution with the least path cost whenever it returns a solution.

. . .

For the A* graph search above, the guarantee combines:

- an **admissible heuristic**;
- **best-cost updates and reopening**;
- a goal test on a **current minimum-f entry**.

The proof concerns the same implementation used by the examples and experiments. The stale-entry check ensures that an outdated entry is not treated as the current best

route to its state. Completeness additionally asks whether the algorithm finds a solution whenever one exists; finite and infinite spaces require different termination assumptions.

Theorem: Cost Optimality

Consider the **finite reachable state-space graph** of the implementation above, with **unit action costs**. Let $h^\star(s)$ be the least remaining cost from state s to a goal.

The heuristic is **admissible**:

$$0 \le h(s) \le h^\star(s), \qquad h(G) = 0 \text{ for every goal } G.$$

With the **best-cost and reopening policy above**, A* returns a **least-cost solution** when one exists, and reports failure otherwise.

The finite, unit-cost 8-puzzle satisfies these assumptions. The result also extends to finite graphs with nonnegative action costs when the implementation adds each actual action cost. If no goal is reachable from a state, its remaining goal cost is infinite; admissibility imposes no finite upper bound there. The heuristic used by the implementation is a fixed, nonnegative estimate for each state.

The policy matters as much as admissibility. Discarding every later path to a state merely because it was generated earlier does not satisfy this theorem.

Proof

1. Suppose A* selects a suboptimal goal.

Let the optimal solution cost be $C^\star$. Suppose the algorithm returns a goal entry G with cost $C > C^\star$.

Because $h(G) = 0$,

$$f(G) = g(G) + h(G) = C.$$

We examine the frontier immediately before that current goal entry is selected.

Proof

2. An optimal-cost continuation remains on the frontier.

Follow an optimal path from the start. Consider its first state that has **not yet been expanded at its optimal prefix cost**.

A **current frontier entry** reaches that state with that optimal prefix cost.

The start enters the frontier with cost zero. Whenever a state on the chosen optimal path is expanded at its optimal prefix cost, generating the next action offers the next state its own optimal prefix cost.

If this improves `best_g`, the new entry is enqueued, even if the state was expanded earlier along a more expensive path. If the same optimal cost was already recorded, its entry is either still waiting or was already expanded at that cost. Continue along the path in the latter case. No cheaper prefix can exist, because replacing this prefix would produce a cheaper solution.

Therefore the first state not yet expanded at its optimal prefix cost has a waiting entry with that cost. It is not stale: `best_g` cannot improve below the optimal prefix cost. Such a state must exist unless an optimal goal has already been selected, in which case the algorithm would already have returned.

An earlier expansion at a worse cost does not break this argument: the strict improvement rule reopens the state when its optimal prefix is discovered.

Proof

3. That frontier entry has a lower bound no larger than $C^\star$.

Call the entry n and its state s. Its stored $g(n)$ is the optimal prefix cost. By admissibility,

$$f(n) = g(n) + h(s) \le g(n) + h^\star(s) = C^\star.$$

The prefix and the cheapest remaining continuation together form an optimal solution.

Proof

4. The minimum-f rule contradicts selecting the suboptimal goal.

The frontier contains a current entry n with

$$f(n) \le C^\star < C = f(G).$$

A* must select an entry with lower f before G. It therefore cannot return a suboptimal goal.

On a finite graph, nonnegative-cost cycles cannot improve a path cost. Accepting only strict improvements prevents endlessly adding equal-cost cycles; only finitely many improving paths can be accepted. The search therefore terminates. If it exhausts the frontier, no solution remains to be found.

On an **infinite** graph, admissibility alone is not a completeness guarantee. A standard sufficient set of assumptions is finite branching, an existing solution of finite cost, and

action costs bounded below by some $\varepsilon > 0$. Since $h \geq 0$, only finitely many candidate paths can have $f \leq C^\star$ under these assumptions. This excludes an infinite succession of zero-cost steps delaying a solution forever. An infinite unsolvable search need not terminate and report failure.

Consistency

A heuristic is **consistent** if every action from s to s' satisfies

$$h(s) \leq c(s, s') + h(s'), \qquad h(G) = 0.$$

Its estimated remaining cost cannot drop by more than the cost of that action. Consequently, along a path,

$$f(n') = g(n) + c(s, s') + h(s') \geq g(n) + h(s) = f(n).$$

Consistency is a local condition on every transition. Applying it repeatedly along a path to a goal shows that the heuristic is admissible. In the 8-puzzle, one unit-cost move can reduce misplaced tiles by at most one, and can reduce Manhattan distance by at most one; both heuristics are consistent.

With a consistent heuristic, a state's first current minimum-f selection already uses a least-cost path to that state. Reopening after expansion is therefore unnecessary. The same `astar` implementation still works; it simply does not need to expand a state again at a better cost.

Consistency does not justify finalizing a state's cost when it is first generated: a cheaper path can still be found before its first expansion.

An admissible but inconsistent heuristic can require reopening. The best-cost policy retains optimality in that case, although repeated expansions may add work. A graph-search variant that permanently closes states after expansion instead relies on consistency for this guarantee. See Berkeley CS 188, Informed Search.

Constructing an 8-Puzzle Heuristic

What information about a board could estimate the number of moves remaining?

Each action costs one, so the heuristic must estimate moves. The examples exclude the blank when counting misplaced tiles or summing numbered-tile distances. Their goal is the standard ordered board with the blank in the lower-right corner.

Can Inversion Count Be a Heuristic?

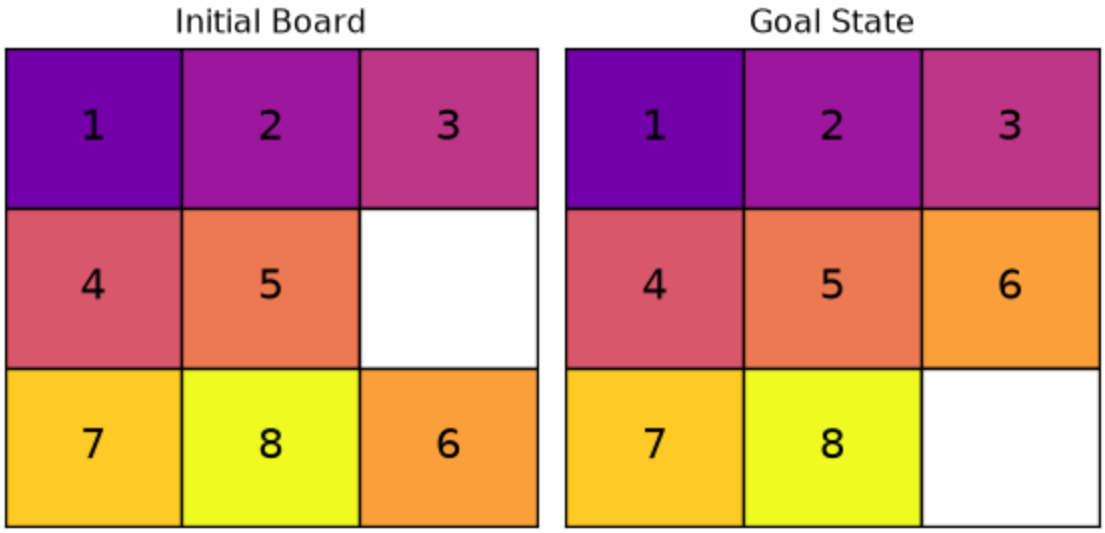
Consider the **raw number of inversions** as $h(s)$.

...

In [4]:
```python
# Define the initial state
initial_board = (1, 2, 3,
                 4, 5, 0,
                 7, 8, 6)

# Define goal state
goal_board = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

# Plot both boards
plt.figure(figsize=(6, 3))
plot_board(initial_board, "Initial Board", 2, 1)
plot_board(goal_board, "Goal State", 2, 2)
plt.tight_layout()
plt.show()
```



- Calculate $h(s)$ and $h^\star(s)$ for this board.
- What does the comparison establish?

The shown board has two inversions: tile 6 follows both 7 and 8. Moving 6 into the blank solves the puzzle in one move. Therefore $h(s) = 2 > 1 = h^\star(s)$, which disproves admissibility of the raw inversion count. The same transition to a goal violates consistency: $2 > 1 + 0$.

On a **3×3 board**, a legal move changes the inversion count by 0 or ±2. A vertical move can move a numbered tile past two other numbered tiles in row-major order. The counterexample shows why raw inversion count is not a lower bound on moves; the possible ±2 change alone is not a proof of inadmissibility.

Parity still serves a different purpose. The board `(1,2,3,4,5,6,0,8,7)` has odd inversion parity and cannot reach the even-parity goal. Solvability and remaining solution

cost are different questions.

Misplaced Tiles

In [5]:
```
def misplaced_tiles_distance(state, goal_state):

    # Count the number of misplaced tiles

    misplaced_tiles = sum(1 for s, g in zip(state, goal_state) if s != g and

    return misplaced_tiles
```

Is this heuristic admissible? The **blank is excluded**.

One legal move changes the position of exactly one numbered tile. It can correct at most one misplaced tile; the other numbered tiles retain their positions. If $h(s)$ tiles are misplaced, any solution therefore needs at least $h(s)$ moves, so $h(s) \leq h^\star(s)$.

The same observation gives consistency: one move can reduce the count by at most one, so $h(s) \leq 1 + h(s')$. The goal has value zero.

Misplaced Tiles on Two Boards

In [6]:
```
GOAL_STATE = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)
SIMPLE_STATE = (1, 2, 3,
                4, 0, 6,
                7, 5, 8)
HARD_STATE = (6, 4, 5,
              8, 2, 7,
              1, 0, 3)

def plot_heuristic_comparison(left, right, goal, heuristics):
    plt.figure(figsize=(8, 2.5))
    for position, state in [(1, left), (3, right)]:
        title = "\n".join(
            f"{name}={heuristic(state, goal)}"
            for name, heuristic in heuristics.items()
        )
        plot_board(state, title, 3, position)
    plot_board(goal, "Goal State", 3, 2)
    plt.tight_layout()
    plt.show()

plot_heuristic_comparison(
    SIMPLE_STATE, HARD_STATE, GOAL_STATE,
    {"misplaced": misplaced_tiles_distance},
)
```



Calling A* with a Heuristic

```
solution_states, expanded = astar(
    SIMPLE_STATE, GOAL_STATE, misplaced_tiles_distance
)
```

One search implementation accepts the heuristic as an argument. It returns a state sequence and the number of non-goal node expansions.

Simple Case: Misplaced Tiles

In [7]:
```
solution_states, expanded = astar(SIMPLE_STATE, GOAL_STATE, misplaced_tiles_
print(f"Solution: {len(solution_states) - 1} moves; nodes expanded: {expande

plt.figure(figsize=(8, 2))
for i, state in enumerate(solution_states):
    plot_board(state, f"Step {i}", len(solution_states), i + 1)
plt.tight_layout()
plt.show()
```

```
Solution: 2 moves; nodes expanded: 2
```



Challenging Case: Misplaced Tiles

In [8]:
```
misplaced_solution, misplaced_expanded = astar(
    HARD_STATE, GOAL_STATE, misplaced_tiles_distance
)
print(f"Solution: {len(misplaced_solution) - 1} moves")
print(f"Nodes expanded: {misplaced_expanded:,}")
```

```
Solution: 25 moves
Nodes expanded: 29,005
```

The returned path includes both endpoints. Use `print_solution(misplaced_solution)` to inspect a complete state sequence.

Manhattan Distance

In [9]:
```python
def manhattan_distance(state, goal_state):
    distance = 0
    size = int(len(state) ** 0.5)
    for num in range(1, len(state)):
        idx1 = state.index(num)
        idx2 = goal_state.index(num)
        x1, y1 = idx1 % size, idx1 // size
        x2, y2 = idx2 % size, idx2 // size
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance
```

$$h_{\text{Manhattan}}(s) = \sum_{t=1}^{8} \left(|x_t - x_t^\star| + |y_t - y_t^\star| \right).$$

The blank is excluded. Why is this a lower bound on the required number of moves?

For tile t, (x_t, y_t) is its current position and $(x_t^\star, y_t^\star)$ is its goal position. The sum includes the eight numbered tiles.

One legal action moves one numbered tile by one square, changing that tile's Manhattan contribution by exactly one in magnitude. At most one unit of the total can be removed per move. Reaching a total of zero therefore takes at least the initial Manhattan sum, proving admissibility. The same observation gives consistency: $h(s) \leq 1 + h(s')$.

The relaxed-problem interpretation makes tiles independent: each numbered tile may move orthogonally without needing the blank, and occupancy constraints are ignored. In that relaxation, each tile can follow a shortest grid path to its target, so the exact total cost is the Manhattan sum. Removing constraints cannot increase the least solution cost, which supplies the lower bound for the real puzzle.

Manhattan Values on Two Boards

In [10]:
```python
plot_heuristic_comparison(
    SIMPLE_STATE, HARD_STATE, GOAL_STATE,
    {"Manhattan": manhattan_distance},
)
```



Comparing the Heuristics

For every reachable state, with the blank excluded,

$$h_{\text{misplaced}}(s) \leq h_{\text{Manhattan}}(s) \leq h^{\star}(s).$$

. . .

- Each misplaced numbered tile contributes at least one to Manhattan distance.
- Manhattan therefore **dominates** misplaced tiles: it is a more informative lower bound.
- Does that guarantee a shorter wall-clock runtime on every instance?

A larger admissible estimate can rule out more unpromising paths. It does not necessarily produce strictly fewer expansions on every problem. Entries with priority equal to the optimal cost can be treated differently by tie rules, and computing a more informative heuristic may cost more per node.

The experiments hold the algorithm, problem instances, and tie policy fixed, and measure non-goal node expansions. They do not measure elapsed runtime. See Berkeley CS 188, Dominance.

Both Estimates on the Same Boards

In [11]:
```
plot_heuristic_comparison(
    SIMPLE_STATE, HARD_STATE, GOAL_STATE,
    {"misplaced": misplaced_tiles_distance, "Manhattan": manhattan_distance}
)
```



Same Misplaced Count, Different Distances

In [12]:
```
comparison_left = (3, 1, 2,
                   4, 5, 6,
                   7, 8, 0)
comparison_right = (8, 2, 3,
                    4, 5, 6,
                    1, 0, 7)
plot_heuristic_comparison(
    comparison_left, comparison_right, GOAL_STATE,
    {"misplaced": misplaced_tiles_distance, "Manhattan": manhattan_distance}
)
```



Both boards have three misplaced numbered tiles, but their Manhattan estimates differ. Misplaced tiles records only whether a tile is in its target position; Manhattan also uses how far it is from that position.

Changing the Heuristic

```
solution_states, expanded = astar(
    SIMPLE_STATE, GOAL_STATE, manhattan_distance
)
```

Only the heuristic argument changes. The frontier, best-cost updates, goal test, and path reconstruction remain the same.

The frontier may contain goal entries. Their heuristic value is zero, so their priority is the cost of the complete path they represent. Another frontier entry with lower priority may

still lead to a cheaper goal. This is why the algorithm tests for a goal after selecting a current minimum-priority entry.

Simple Case: Manhattan

In [13]:
```
solution_states, expanded = astar(SIMPLE_STATE, GOAL_STATE, manhattan_distar
print(f"Solution: {len(solution_states) - 1} moves; nodes expanded: {expande

plt.figure(figsize=(8, 2))
for i, state in enumerate(solution_states):
    plot_board(state, f"Step {i}", len(solution_states), i + 1)
plt.tight_layout()
plt.show()
```

```
Solution: 2 moves; nodes expanded: 2
```



Challenging Case: Manhattan

In [14]:
```
manhattan_solution, manhattan_expanded = astar(
    HARD_STATE, GOAL_STATE, manhattan_distance
)
print(f"Solution: {len(manhattan_solution) - 1} moves")
print(f"Nodes expanded: {manhattan_expanded:,}")
```

```
Solution: 25 moves
Nodes expanded: 2,255
```

The returned path includes both endpoints. Use `print_solution(manhattan_solution)` to inspect a complete state sequence.

Paired Experiments

- Use the **same 1,000 solvable boards** with both heuristics.
- Keep the **same A* implementation and tie policy**.
- Retain both solution lengths and compare **node expansions**.

The heap entries are `(f, g, state, path)`. Equal `f` values are broken first by smaller `g`, then lexicographically by the state tuple. This deterministic policy is shared by both heuristic runs; it may differ from a FIFO queue's within-depth order.

The local random generator makes the experiment independent of random draws in earlier examples. The two length arrays allow a direct correctness check rather than overwriting one result. Agreement between the two heuristic runs alone is not a proof of optimality; a useful implementation check also compares a bounded sample against BFS.

In [15]:
```
experiment_rng = random.Random(58)
experiment_states = []
while len(experiment_states) < 1000:
    state = tuple(experiment_rng.sample(range(9), 9))
    if is_solvable(state):
        experiment_states.append(state)

moves_misplaced = []
moves_manhattan = []
expanded_misplaced = []
expanded_manhattan = []

for initial_state in experiment_states:
    path_misplaced, count_misplaced = astar(
        initial_state, GOAL_STATE, misplaced_tiles_distance
    )
    path_manhattan, count_manhattan = astar(
        initial_state, GOAL_STATE, manhattan_distance
    )
    moves_misplaced.append(len(path_misplaced) - 1)
    moves_manhattan.append(len(path_manhattan) - 1)
    expanded_misplaced.append(count_misplaced)
    expanded_manhattan.append(count_manhattan)

assert moves_misplaced == moves_manhattan
print(f"Paired solvable boards: {len(experiment_states)}")
print("Both heuristics return the same solution cost on every board.")
print(f"Mean expansions, misplaced tiles: {np.mean(expanded_misplaced):,.1f}
print(f"Mean expansions, Manhattan:       {np.mean(expanded_manhattan):,.1f}
```

```
Paired solvable boards: 1000
Both heuristics return the same solution cost on every board.
Mean expansions, misplaced tiles: 16,469.7
Mean expansions, Manhattan:       1,708.2
```

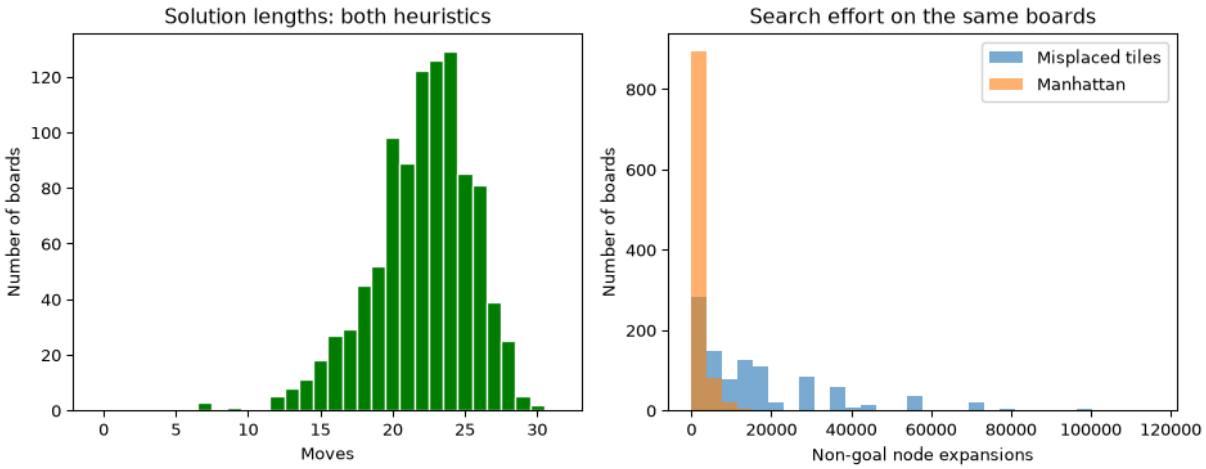
1,000 Paired Searches

In [16]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].hist(moves_misplaced, bins=np.arange(-0.5, 32.5), color='green', edg
axes[0].set_title("Solution lengths: both heuristics")
axes[0].set_xlabel("Moves")
axes[0].set_ylabel("Number of boards")

bins = np.histogram_bin_edges(expanded_misplaced + expanded_manhattan, bins=
axes[1].hist(expanded_misplaced, bins=bins, alpha=0.6, label="Misplaced tile
axes[1].hist(expanded_manhattan, bins=bins, alpha=0.6, label="Manhattan")
axes[1].set_title("Search effort on the same boards")
```

```
axes[1].set_xlabel("Non-goal node expansions")
axes[1].set_ylabel("Number of boards")
axes[1].legend()
plt.tight_layout()
plt.show()
```

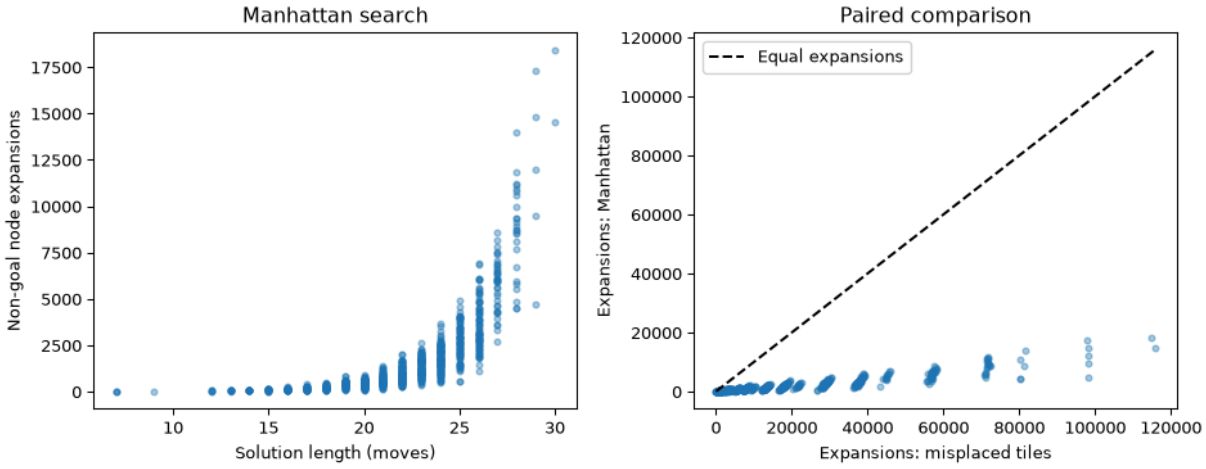


Comparing Search Effort

In [17]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].scatter(moves_manhattan, expanded_manhattan, s=12, alpha=0.4)
axes[0].set_title("Manhattan search")
axes[0].set_xlabel("Solution length (moves)")
axes[0].set_ylabel("Non-goal node expansions")

axes[1].scatter(expanded_misplaced, expanded_manhattan, s=12, alpha=0.4)
limit = max(max(expanded_misplaced), max(expanded_manhattan))
axes[1].plot([0, limit], [0, limit], '--', color='black', label="Equal expar
axes[1].set_title("Paired comparison")
axes[1].set_xlabel("Expansions: misplaced tiles")
axes[1].set_ylabel("Expansions: Manhattan")
axes[1].legend()
plt.tight_layout()
plt.show()
```



Each point on the right represents one board. A point below the diagonal means fewer expansions with Manhattan on that board.

Empirical Validation

- With unit action costs, **BFS supplies a minimum-move reference**.
- Compare BFS and A* with Manhattan distance on **25,000 unique solvable boards**.
- Check solution lengths and record **non-goal node expansions**.
- Agreement increases confidence in the code; **it is not a proof**.

Download the Python validation script and run it from the lecture folder:

```
python3 validate_astar.py 4106 25000
```

Keep the generated `astar_validation_runs-25000_seed-4106.csv` beside the notebook. The next slide reads these saved results.

The mathematical proof establishes A*'s cost optimality under its assumptions. This experiment checks whether our implementation behaves as expected on a substantial sample. A discrepancy is evidence of an implementation error; agreement does not rule out every error, especially because BFS and A* share the successor generator and goal test.

The script compares solution lengths, reports disagreements on standard error, and retains both expansion counts in the CSV. Additional checks can verify that returned paths use legal moves and reach the goal. Different optimal paths may have the same length.

The seed makes the experiment reproducible. Duplicate boards are skipped before either search, and a completed run contains 25,000 distinct boards from the 181,440 solvable configurations. Each completed row is saved immediately, so the results slide may show a partial experiment while the script is still running. Generating the data is a separate, manual step; reading the CSV does not rerun the searches.

Saved Results: BFS versus A*

In [18]:
```
import csv
import os
from pathlib import Path
from IPython.display import Markdown, display

number_translation = str.maketrans({",": ",", ".": "."})
def format_number(value, decimals=0):
    return f"{value:,.{decimals}f}".translate(number_translation)

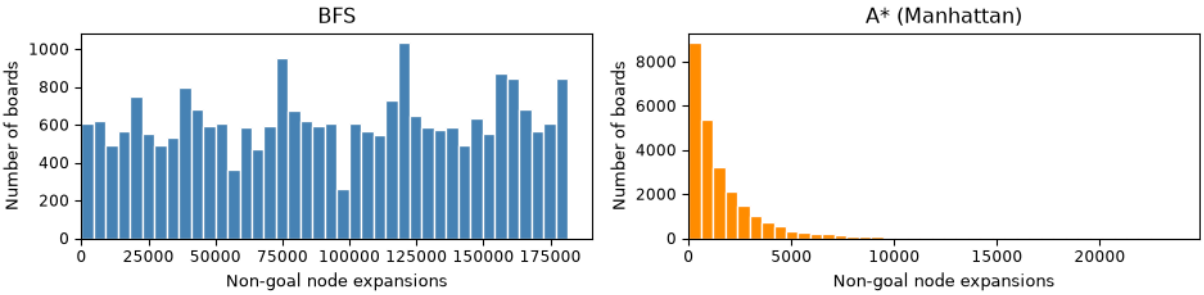
CSV_NAME = "astar_validation_runs-25000_seed-4106.csv"
csv_path = Path(CSV_NAME)
```

```
project_dir = os.environ.get("QUARTO_PROJECT_DIR")
if not csv_path.is_file() and project_dir:
    csv_path = Path(project_dir) / "lectures" / "17" / CSV_NAME

if not csv_path.is_file():
    display(Markdown(
        "**Saved results are not available yet.** Run this once beside the r
        "then rerun the notebook or render the slides:\n\n"
        "```bash\npython3 validate_astar.py 4106 25000\n```"
    ))
else:
    # Read a complete prefix even while the experiment is appending rows.
    csv_text = csv_path.read_text(encoding="utf-8")
    complete_text = csv_text[:csv_text.rfind("\n") + 1]
    records = list(csv.DictReader(complete_text.splitlines()))

    if not records:
        display(Markdown("**The CSV exists, but no completed results are ava
    else:
        unique_boards = len({row["board"] for row in records})
        mismatches = sum(
            row["bfs_moves"] == "" or row["astar_moves"] == ""
            or row["bfs_moves"] != row["astar_moves"]
            for row in records
        )
        status = "Partial results. " if len(records) < 25000 else ""
        display(Markdown(
            f"**{status}{format_number(len(records))} completed comparisons*
            f"({format_number(unique_boards)} unique boards); **{format_numb
        ))

        methods = [
            ("BFS", [int(row["bfs_expanded"]) for row in records], "steelblu
            ("A* (Manhattan)", [int(row["astar_expanded"]) for row in record
        ]
        fig, axes = plt.subplots(1, 2, figsize=(10, 2.6))
        table = ["| Method | Mean | Median | Minimum | Maximum |", "|---|---
        for ax, (name, expanded, color) in zip(axes, methods):
            bins = np.linspace(0, max(1, max(expanded)), 41)
            ax.hist(expanded, bins=bins, color=color, edgecolor="white")
            ax.set_title(name)
            ax.set_xlabel("Non-goal node expansions")
            ax.set_ylabel("Number of boards")
            ax.set_xlim(0, max(1, max(expanded)) * 1.05)
            ax.ticklabel_format(style="plain", axis="x")
            ax.tick_params(labelsize=9)
            table.append(
                f"| {name} | {format_number(np.mean(expanded), 1)} | {format
                f"| {format_number(min(expanded))} | {format_number(max(expa
            )
        plt.tight_layout()
        plt.show()
        display(Markdown("\n".join(table)))
```



The histograms use **different horizontal ranges** to reveal both distributions. Compare the numerical scales and statistics.

Each row pairs the two algorithms on the same initial board. The table reports the mean, median, minimum and maximum numbers of non-goal expansions. The selected goal is excluded; stale A* entries are excluded, while any actual repeated expansion would be counted.

The histograms use separate horizontal ranges so that the shape of each distribution remains visible. A narrow concentration at small counts indicates that many boards require little search effort; a long right tail identifies a smaller group of expensive cases. Compare the axis values and the table before drawing conclusions about relative effort. Expansion counts measure search work, rather than elapsed runtime or total memory.

The sample size and any solution-length failures are computed from the completed CSV rows, rather than inferred from the requested run count in the filename. Partial results should not be described as a completed 25,000-board experiment. The experiment and the proof answer complementary questions about the code and the algorithm.

Exploration: Validate Optimality

BFS returns a minimum-move solution when every action costs one.

1. Generate a bounded sample of solvable 8-puzzle boards.
2. Find a BFS solution and an A* solution for each board.
3. Check that each path is legal, reaches the goal, and has the same number of moves.
4. Repeat with both admissible heuristics.

BFS supplies an independent reference for solution cost; it need not follow the same path as A*. Multiple optimal solutions can exist. Keep the validation sample manageable because BFS can visit a large fraction of the reachable state space for each board.

Exploration: A Zero Heuristic

With $h(s) = 0$, A* orders its frontier by $g(n)$ alone: it becomes **uniform-cost search**.

. . .

When all action costs are one, both uniform-cost search and BFS expand nodes in **nondecreasing depth**.

Must their expansion order and expansion counts be identical?

They agree on minimum solution cost, but their order within a depth can differ. The heap's tie rule, successor order, duplicate policy, and goal-test location affect the order and the work completed before the goal is selected. Graph search avoids repeated states; it does not enumerate every possible path.

Results on the Lecture 16 Board

Method	Solution length	Non-goal expansions
BFS	25 moves (optimal)	145,605
DFS	1,157 moves	1,187
A* with Manhattan	25 moves (optimal)	2,255

These results concern one fixed board, successor order, and tie policy. Expansion counts do not measure wall-clock runtime or total memory.

The BFS and DFS baselines come from Lecture 16. The A* result uses the single corrected implementation above. All methods return paths, but only BFS with equal action costs and A* under the stated conditions guarantee a least-cost solution.

Measuring Performance

- **Completeness:** Does the algorithm find a solution whenever one exists?
- **Cost optimality:** Does it return a solution with the least path cost?

On a finite reachable state-space graph, the BFS and DFS implementations with repeated-state detection are complete. The corrected A* implementation is also complete under the theorem's assumptions. They can exhaust the frontier and return failure when no goal is reachable.

In infinite spaces, finding an existing solution and terminating on every unsolvable problem are different requirements. DFS can follow an infinite branch. Standard A* completeness assumptions include finite branching, a finite-cost solution, and action costs bounded below by a positive constant.

BFS is cost optimal for equal action costs. DFS has no general cost-optimality guarantee. A* relies on admissibility and the specified best-cost, reopening, and goal-selection policies.

Time and Space Complexity

- **State-space graph:** use the number of states V and actions E.
- **Search tree:** use branching factor b and solution depth d.
- **Actual implementation:** also account for heaps, heuristic evaluation, and stored paths.

The examples report **node expansions**. A complete performance comparison must also consider the work and memory used by each expansion.

With constant-time state operations and constant-size per-node records, ordinary graph BFS takes $\mathcal{O}(V + E)$ time and $\mathcal{O}(V)$ storage. The lecture code copies full paths into frontier entries, which adds work and storage proportional to their lengths; these costs must not be silently omitted when describing the implementation.

For tree-style BFS with goal testing on generation, the familiar bound is $\mathcal{O}(b^d)$ generated nodes. With goal testing on removal, as in our code, nodes at depth $d + 1$ may be generated before a depth-d goal is selected, giving $\mathcal{O}(b^{d+1})$ generated or stored occurrences in the corresponding worst-case tree analysis. State-space graph bounds and search-tree bounds count different objects.

A* adds priority-queue operations and heuristic evaluations. With inconsistent admissible heuristics it may also expand a state again after an improved path, so the expansion count can exceed the number of distinct expanded states. A small frontier does not remove the memory retained in best-cost records and copied paths.

Videos by Sebastian Lague

- A* Pathfinding (E01: algorithm explanation) posted on 2014-12-16.
- A* Pathfinding (E02: node grid) posted on 2014-12-18.
- A* Pathfinding (E03: algorithm implementation) posted on 2014-12-19.
- A* Pathfinding (E04: heap optimization) posted on 2014-12-24.
- A* Pathfinding (E05: units) posted on 2015-01-06.
- A* Pathfinding (E06: weights) posted on 2015-01-11.
- A* Pathfinding (E07: smooth weights) posted on 2016-12-30.
- A* Pathfinding (E08: path smoothing 1/2) posted on 2017-01-31.
- A* Pathfinding (E09: path smoothing 2/2) posted on 2017-01-31.
- A* Pathfinding (E10: threading) posted on 2017-02-03.
- A* Pathfinding Tutorial (Unity) (play list)

A resource dedicated to $A^\star$

- Amit's A* Pages

Epilogue

Summary

- Best-first priorities distinguish **greedy search**, **uniform-cost search**, and **A***.
- A* combines **known path cost** with an **estimated remaining cost**.
- **Best-cost updates, reopening, and goal selection** connect the implementation to the optimality proof.
- **Admissibility, consistency, and dominance** justify and compare the puzzle heuristics.

An A* frontier entry represents a particular path. `best_g` records the cheapest discovered route to each state, while stale entries can remain in the heap until they are removed. An improved route may reopen a state. These policies preserve the optimal-path frontier entry used in the proof.

Misplaced tiles and Manhattan distance exclude the blank and are both admissible and consistent for unit-cost 8-puzzle moves. Manhattan dominates misplaced tiles, but computation costs and tie choices still matter when measuring performance.

On the fixed Lecture 16 board, BFS and A* with Manhattan both return 25 moves, with 145,605 and 2,255 non-goal expansions respectively. DFS returns a longer path after fewer expansions. The paired experiment compares both heuristics on identical boards and retains their solution costs separately.

Next Session

- **Quiz 2:** November 18, 2026. See the course FAQ.
- After the quiz: Local Search, Lecture 19.

References

Hart, Peter E., Nils J. Nilsson, and Bertram Raphael. 1968. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths.” *IEEE Transactions on Systems Science and Cybernetics* 4 (2): 100–107. https://doi.org/10.1109/tssc.1968.300136.

Russell, Stuart, and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson. http://aima.cs.berkeley.edu/.

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

School of Electrical Engineering and **Computer Science** (EE**CS**)

University of Ottawa